

Errors, File Handling & Modules

Types of Errors, try/except/finally, Reading & Writing Files, and Python’s Built-In Toolbox

Session 11

What We're Covering Today

- Types of errors — syntax, logical, and runtime, and how to tell them apart
- Exception handling — try, except, else, finally, and raising your own errors
- File handling — opening, reading, writing, and safely closing files
- What a module actually is, and how import really works
- Built-in modules — datetime, time, random, and a few more worth knowing
- Today's tasks and two mini projects combining everything



PART 1

Types of Errors

Before you can handle an error, you need to know which of the three kinds you're looking at

What Is an Error?



Three different failure modes

- An "error" isn’t one thing — Python code can fail in three fundamentally different ways, and the fix for each is completely different
- Syntax errors — the code is written incorrectly and can’t even start running
- Logical errors — the code runs fine, but produces the wrong answer
- Runtime errors (exceptions) — the code is valid and starts running, but something goes wrong partway through

Syntax Errors — Caught Before the Code Even Runs

- A syntax error means the code breaks Python's grammar rules — a missing colon, an unclosed bracket, a mismatched quote
- Python catches these before executing a single line — you get a SyntaxError and the program never starts
- These are usually the easiest to fix — the traceback points directly at the exact line and often the exact character

```
def greet(name)      # missing colon
  print(f"Hi {name}")

 File "script.py", line 1
  def greet(name)
               ^
SyntaxError: expected ':'
```



Logical Errors — The Code Runs, But the Answer Is Wrong

- A logical error means the code runs with no crash at all — Python has no way to detect it, because nothing is technically "wrong"
- The bug is in your thinking, not your syntax — the wrong formula, a swapped condition, an off-by-one loop
- These are the hardest to find precisely because Python never complains — you only notice from incorrect output

```
def average(a, b):
    return a + b / 2      # BUG: should be (a + b) / 2

average(10, 20)
# 20.0   <- wrong! division happened before addition
# Expected: 15.0

# No error message anywhere — you have to notice
# the output is wrong yourself
```

Common Runtime Errors (Exceptions) — Part 1

ZeroDivisionError

10 / 0 — dividing by zero, always fatal, never a valid result

IndexError

my_list[10] on a 3-item list — asking for a position that doesn’t exist

KeyError

my_dict["missing"] — asking for a dictionary key that was never set

Common Runtime Errors (Exceptions) — Part 2

TypeError

"5" + 5 — combining incompatible types the operation can’t handle

ValueError

int("abc") — the right type, but a value that can’t be converted or used

FileNotFoundError

open("missing.txt") — trying to open a file that isn’t where you said it was

Syntax vs. Logical vs. Runtime — Side by Side

Syntax Error

Caught before running. Program never starts. Fix: correct the grammar Python is pointing at.

Logical Error

No crash at all. Wrong output. Fix: trace your logic by hand, check the actual math or condition.

Runtime Error

Program starts, then crashes partway through. Fix: handle it with try/except, or prevent the bad input.

Practice Set — Types of Errors

Try these yourself first — no code shown, that's the point

1. Write a function with a deliberate missing colon, run it, and read the exact SyntaxError message

2. Write a function that calculates a percentage but has a deliberate logical bug — then find and fix it by tracing the math by hand

3. Trigger a ZeroDivisionError, an IndexError, and a KeyError on purpose, and read each traceback carefully

4. For each of the three errors above, say out loud which category (syntax/logical/runtime) it belongs to and why



PART 2

Exception Handling

You can’t prevent every runtime error — but you can control what happens when one occurs

What Is Exception Handling?



Fail on purpose, not by accident

- Exception handling lets you catch a runtime error when it happens, and decide what your program does next — instead of crashing outright
- Without it, one bad input from a user can end the entire program, even if everything else was working fine
- try/except is the core tool — "try this risky code, and if it fails in a specific way, do this instead"
- This is essential in any real program that takes input from users, files, or the internet — all places where things go wrong constantly



try / except — The Basics

- Code inside try runs first — if it completes with no error, except is skipped entirely
- If an error occurs inside try, Python immediately jumps to except and runs that block instead
- The rest of the program continues normally afterward — the crash is contained, not fatal

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
except:
    print("Something went wrong — try again")

# Input: "abc"    -> Something went wrong — try again
# Input: "0"      -> Something went wrong — try again
# Input: "5"      -> 2.0
```

Catching Specific Exceptions

- A bare except catches everything — which also hides bugs you didn’t expect and didn’t mean to catch
- Naming the exact exception type lets you respond differently to different problems, and lets unexpected errors still surface normally
- Multiple except blocks can stack — Python checks them top to bottom and runs the first one that matches

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
except ValueError:
    print("That's not a valid number")
except ZeroDivisionError:
    print("Can't divide by zero")
```



Accessing the Exception Object with as

- except SomeError as e captures the actual exception object, not just its type — e holds the real error details
- str(e) or just printing e shows Python's own explanation of exactly what went wrong
- This is invaluable for logging — you keep the program running, but still record precisely what happened

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error occurred: {e}")
    # Error occurred: division by zero

try:
    int("abc")
except ValueError as e:
    print(f"Error occurred: {e}")
    # Error occurred: invalid literal for int() with base 10: 'abc'
```

else — Runs Only When Nothing Went Wrong

- An else block runs only if the try block completed with no exception at all
- This separates "the risky part" (in try) from "what to do once it definitely succeeded" (in else) — keeping each block's purpose clear
- It's optional, and less commonly used than try/except, but it makes success-only logic explicit rather than just tacked onto the end of try

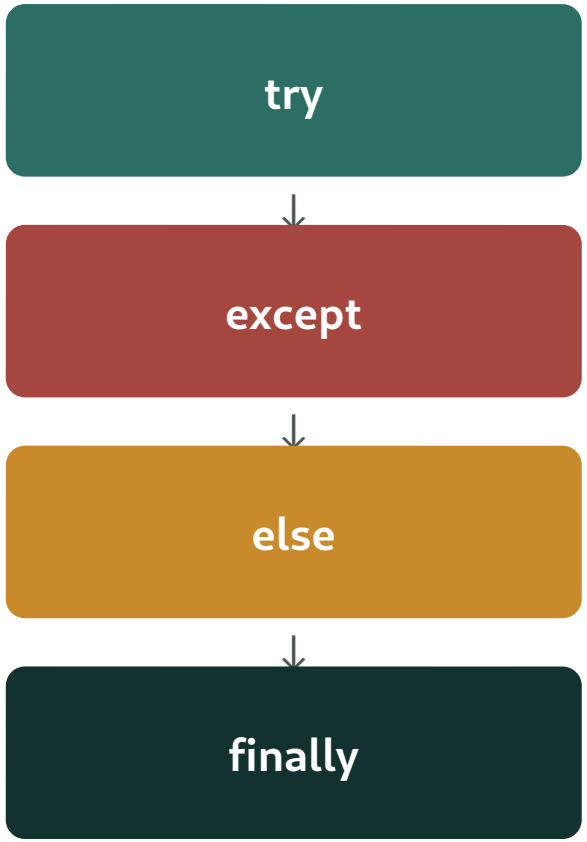
```
try:
    number = int(input("Enter a number: "))
except ValueError:
    print("That's not a valid number")
else:
    print(f"Great, you entered {number}")   # only if NO error
```

finally — Always Runs, No Matter What

- finally runs unconditionally — whether the try succeeded, an exception was caught, or even if the exception wasn’t caught at all
- This is the right place for cleanup that absolutely must happen — closing a file, releasing a resource, closing a connection
- Even a return inside try or except doesn’t skip finally — it always executes before the function actually returns

```
try:
    number = int(input("Enter a number: "))
except ValueError:
    print("Invalid number")
finally:
    print("This always runs")
```

try / except / else / finally — The Full Flow



Runs first, always

Runs ONLY if an error happened in try

Runs ONLY if NO error happened in try

Runs ALWAYS — error or not, return or not

except and else are mutually exclusive for a given run — finally runs no matter which path was taken.



raise — Triggering Your Own Exceptions

- raise lets your own code deliberately trigger an exception — not just react to Python's built-in ones
- This is how you enforce your own rules: "this value is technically valid Python, but invalid for my program"
- You can raise a built-in exception type with a custom message, exactly like Python's own errors

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount

withdraw(100, 500)
# ValueError: Insufficient funds

try:
    withdraw(100, 500)
except ValueError as e:
    print(f"Transaction failed: {e}")
```



Custom Exception Classes

- You can define your own exception types by inheriting from Exception — giving errors in your program their own specific, meaningful names
- This lets calling code catch exactly your custom error, separately from every other kind of error
- Real-world codebases do this constantly — InsufficientFundsError says far more than a generic ValueError

```
class InsufficientFundsError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(f"Need {amount}, have {balance}")
    return balance - amount

try:
    withdraw(100, 500)
except InsufficientFundsError as e:
    print(f"Blocked: {e}")   # Blocked: Need 500, have 100
```

Real-World Use: Safely Validating User Input

- A while loop combined with try/except is the standard pattern for "keep asking until the input is actually valid"
- The loop never trusts the user's first attempt — it keeps trying until a genuinely valid value gets through
- This exact pattern shows up in signup forms, CLI tools, and configuration wizards everywhere

```
def get_valid_age():
    while True:
        try:
            age = int(input("Enter your age: "))
            if age < 0:
                raise ValueError("Age can't be negative")
            return age
        except ValueError as e:
            print(f"Invalid input ({e}) — try again")
```

Practice Set — Exception Handling

Try these yourself first — no code shown, that's the point

1. Write a safe_divide(a, b) function that catches ZeroDivisionError and returns None instead of crashing

2. Write a loop that keeps asking for a number until valid, using try/except/else

3. Add a finally block that prints "Attempt finished" after every single try, success or failure

4. Create a custom NegativeValueError and raise it from a function that rejects negative input



PART 3

File Handling

Everything your program does disappears when it ends — unless you save it to a file

Why File Handling Matters



Data that survives the program ending

- Every variable in a running program lives only in memory — the moment the program ends, that data is gone
- Files let a program persist data — logs, saved settings, user records — across separate runs
- This is also how programs actually exchange data with each other — config files, exported reports, data feeds
- File handling is where error handling becomes essential — missing files, permission issues, and full disks are all things that go wrong constantly



open() and File Modes

- open(filename, mode) is the starting point for any file operation — it returns a file object you then read from or write to
- "r" (read, default) fails if the file doesn't exist. "w" (write) creates or completely overwrites the file. "a" (append) adds to the end without erasing what's there
- "x" (exclusive create) fails if the file already exists — useful to avoid accidentally overwriting something

```
open("notes.txt", "r")    # read — error if missing
open("notes.txt", "w")    # write — overwrites everything
open("notes.txt", "a")    # append — adds to the end
open("notes.txt", "x")    # create — error if it already
exists
open("notes.txt", "r+")   # read AND write, file must
exist
```



Reading Files

- read() pulls in the entire file as one single string — fine for small files, wasteful for huge ones
- readline() reads just one line at a time, remembering where it left off on the next call
- readlines() returns every line as a list of strings; looping directly over the file object does the same thing, one line at a time, more memory-efficiently

```
file = open("notes.txt", "r")

file.read()          # entire file as one string
file.readline()      # just the next line
file.readlines()     # list of every line

for line in file:            # most common pattern
    print(line.strip())      # .strip() removes the
trailing \n

file.close()
```



Writing Files

- write() writes a string exactly as given — it does not add a newline automatically, unlike print()
- writelines() writes a list of strings back to back — you still need to include \n yourself if you want separate lines
- Mode "w" starts the file empty every time you open it — use "a" if you want to keep what was already there

```
file = open("notes.txt", "w")
file.write("First line\n")
file.write("Second line\n")
file.close()

lines = ["Third line\n", "Fourth line\n"]
file = open("notes.txt", "a")   # append, don't erase
file.writelines(lines)
file.close()
```



with — The Safer Way to Handle Files

- Forgetting file.close() leaves the file locked or data unsaved — an easy, common mistake with manual open/close
- with open(...) as file: automatically closes the file when the block ends — even if an error occurs inside it
- This is the standard, professional way to work with files in Python — you'll rarely see manual close() in real code

```
# Manual way — easy to forget close(), unsafe on errors
file = open("notes.txt", "r")
content = file.read()
file.close()

# The right way — closes automatically, even on error
with open("notes.txt", "r") as file:
    content = file.read()
# file is already closed here, guaranteed
```



Checking Whether a File Exists First

- os.path.exists() checks whether a file or folder is actually there before you try to use it
- This lets you avoid a FileNotFoundError entirely, rather than catching one after the fact — both are valid, depending on the situation
- In real projects, both patterns are common: check-first for expected-missing files, try/except for genuinely unexpected failures

```
import os

if os.path.exists("notes.txt"):
    with open("notes.txt", "r") as file:
        print(file.read())
else:
    print("File doesn't exist yet")
```



Real-World Use: A Safe Log Writer

- Combining with, append mode, and try/except is exactly how real applications write to log files
- The function never crashes the whole program just because logging failed — it fails safely and reports the problem
- This is the same pattern behind almost every real logging system you'll ever use

```
def write_log(message):
    try:
        with open("app.log", "a") as file:
            file.write(message + "\n")
    except OSError as e:
        print(f"Could not write to log: {e}")

write_log("User logged in")
write_log("Payment processed")
```

Practice Set — File Handling

Try these yourself first — no code shown, that's the point

1. Write a script that creates a file, writes three lines to it, then reads and prints them back

2. Use with to open a file for reading, and confirm it’s closed afterward by checking file.closed

3. Write a function that appends a new line to a file only if the file already exists, using os.path.exists()

4. Wrap a file read in try/except to handle FileNotFoundError gracefully, with a helpful message



PART 4

Modules

You’ve been using modules since your very first import — now let’s understand what they actually are

What Is a Module?



Just a .py file, reusable

- A module is simply a Python file — any .py file can be imported and reused inside another one
- Python ships with a huge standard library of pre-written modules — datetime, random, math, os, and hundreds more — so you don’t have to write everything from scratch
- import brings a module’s functions, classes, and variables into your own file, under that module’s name
- This is the entire foundation of code reuse in Python — organizing related code into files, and pulling in only what you need



Import Styles

- import module_name brings in the whole module — you access things with module_name.thing
- from module_name import thing brings in just that one piece directly — no prefix needed
- import module_name as alias gives it a shorter name to type — extremely common with popular libraries

```
import random
random.randint(1, 10)          # must prefix with random.

from random import randint
randint(1, 10)                 # no prefix needed

import random as rnd
rnd.randint(1, 10)              # shorter alias
```

Creating Your Own Module

- Any .py file you write is automatically a module — there's no special syntax needed to "make" one
- A file named helpers.py containing functions can be imported from another file in the same folder with import helpers
- This is exactly how you split a growing project into organized, reusable files instead of one giant script

```
# helpers.py
def greet(name):
    return f"Hello, {name}!"

PI = 3.14159

# main.py (same folder)
import helpers

print(helpers.greet("Ahmed"))   # Hello, Ahmed!
print(helpers.PI)               # 3.14159
```



A Few Standard Library Modules Worth Knowing



datetime

Working with dates, times, and formatting them for display or storage



time

Pausing execution, and measuring how long code takes to run



random

Random numbers, random choices, and shuffling — games, sampling, simulations



math

Square roots, powers, trig, constants like pi — beyond basic arithmetic



os

Talking to the operating system — file paths, checking what exists, environment info

datetime — Working With Dates and Times

- datetime.now() gets the current date and time, right down to the microsecond
- You can do arithmetic directly with dates — adding a timedelta shifts a date forward or backward by days, hours, or more
- strftime() formats a date into exactly the string layout you want, using format codes like %Y, %m, %d

```
from datetime import datetime, timedelta

now = datetime.now()

print(now) # 2026-09-21 21:50:01.123456

tomorrow = now + timedelta(days=1)

print(tomorrow.strftime("%Y-%m-%d")) # 2026-09-22

print(now.strftime("%B %d, %Y")) # September 21, 2026
```

time — Pausing and Measuring

- time.sleep(seconds) pauses execution for that many seconds — useful for rate-limiting, animations, or retry delays
- time.time() returns the current moment as a number (seconds since a fixed reference point) — subtracting two readings gives elapsed time
- This exact pattern — read time, do work, read time again, subtract — is the standard way to measure how long code takes

```
import time

print("Starting...")
time.sleep(2)          # pause for 2 seconds
print("Done waiting")

start = time.time()
total = sum(range(1_000_000))    # some work
end = time.time()
print(f"Took {end - start:.4f} seconds")
```



random — Randomness, Choices, and Shuffling

- random.randint(a, b) gives a random whole number, including both endpoints
- random.choice(sequence) picks one random item from a list, tuple, or string — random.sample() picks several without repeats
- random.shuffle(list) rearranges a list randomly, in place — it modifies the original list rather than returning a new one

```
import random

random.randint(1, 6)                  # a die roll, 1 to 6
random.choice(["rock", "paper", "scissors"])  # one random pick
random.sample(range(1, 50), 6)        # 6 unique lottery numbers

cards = ["A", "K", "Q", "J"]
random.shuffle(cards)                 # shuffles cards in place
```

Practice Set — Modules

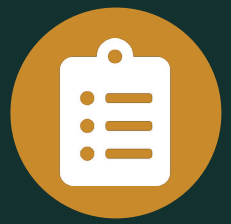
Try these yourself first — no code shown, that's the point

1. Print today's date formatted as "DD/MM/YYYY" using datetime and strftime()

2. Write a function that times how long a loop of 100,000 iterations takes, using time.time()

3. Simulate rolling two dice using random.randint() and print their sum

4. Create your own greetings.py module with two functions, then import and use it from a separate script



INDUSTRY-STYLE TASKS

Build the Things You'll Actually Be Asked to Build

Not toy examples — safe file writers, validated input loops, and reusable modules are everyday, real-world work

Today's Tasks — Part A: Errors & Exception Handling

Small tasks — do them in order

1. Write a divide(a, b) function that catches ZeroDivisionError and TypeError separately, with a specific message for each.

2. Build a get_valid_email() loop that keeps asking until the input contains "@", raising and catching a custom InvalidEmailError.

3. Write a function using try/except/else/finally together, and add a comment explaining exactly when each block runs.

Today's Tasks — Part B: File Handling

Small tasks — do them in order

1. Write a function that appends a timestamped entry to a "diary.txt" file every time it’s called, using with and append mode.

2. Write a function that reads a file of numbers (one per line) and returns their sum, wrapped in try/except for FileNotFoundError.

3. Write a word_count(filename) function that reads a file and returns how many words it contains in total.

Today's Tasks — Part C: Modules

Small tasks — do them in order

1. Write a function that returns how many days are left until a given future date, using datetime.

2. Build a simple "password generator" using random.choice() over letters, digits, and symbols.

3. Create a stopwatch.py module with start_timer() and stop_timer() functions built on the time module, then use it from another script.



PART H

Put It All Together

Two projects combining error handling, file I/O, and Python's built-in modules into one cohesive task

Mini Project 1: Safe File-Based Logger



Errors + files + datetime

- Build a log_event(message, level="INFO") function that appends timestamped entries to "app.log", using datetime for the timestamp
- Wrap every file operation in try/except so a full disk or permissions issue never crashes the calling program
- Add a read_logs(level=None) function that reads the file back and optionally filters by log level
- This mirrors how real logging utilities are built in production code, before reaching for a full logging library

Mini Project 2: Number Guessing Game With Stats



Random + time + error handling

- Use random.randint() to pick a secret number, and time.time() to track how long the player takes to guess it
- Wrap every guess in try/except so non-numeric input never crashes the game — just asks again
- When the player wins, save their name, number of guesses, and time taken as a new line in "scores.txt"
- Add a "view leaderboard" option that reads scores.txt back and prints the fastest three attempts



RECAP

Errors are unavoidable — try/except/finally, safe file handling, and Python's built-in modules are how real programs survive them.

Next session: Advanced Python — comprehensions, generators, and regular expressions